



**SIMATS**  
**ENGINEERING**



**SIMATS**  
Saveetha Institute of Medical And Technical Sciences  
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

# **ANALYTICAL STUDY AND IMPLEMENTATION OF BRUTE FORCE ALGORITHMS FOR SELECTION SORT AND SEQUENTIAL SEARCH**

*Submitted in the partial fulfilment for the course of*

**CSA0603 - Design and Analysis of Algorithms**

to the award of the degree of

**BACHELOR OF TECHNOLOGY**

**IN**

**B.Tech CSE - BioSciences**

Submitted by

S. Susmitha (192571068)

Saswadhana P (192521029)

Selvamagal M (192472013)

Under the Supervision of

Dr. P. Solainayagi

Saveetha School of Engineering

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

September 2026

## COMMON COURSE ASSIGNMENT FORMAT

### A. Assignment Information

|                                       |                                                                                                                                                                                                                                                                                                      |
|---------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Department:</b>                    | Computer Science and BioSciences                                                                                                                                                                                                                                                                     |
| <b>Programme:</b>                     | B.Tech (CSE - BIO)                                                                                                                                                                                                                                                                                   |
| <b>Course Code &amp; Course Name:</b> | CSA0603 - Design and Analysis of Algorithms                                                                                                                                                                                                                                                          |
| <b>Academic Year / Batch:</b>         | 2nd Year                                                                                                                                                                                                                                                                                             |
| <b>Faculty Name:</b>                  | Dr. P. Solainayagi                                                                                                                                                                                                                                                                                   |
| <b>Assignment Title:</b>              | Analytical Study and Implementation of Brute Force Algorithms for Selection Sort and Sequential Search                                                                                                                                                                                               |
| <b>Date of Issue:</b>                 | 01/09/2026                                                                                                                                                                                                                                                                                           |
| <b>Date of Submission:</b>            | 15/09/2026                                                                                                                                                                                                                                                                                           |
| <b>Maximum Marks:</b>                 | 100                                                                                                                                                                                                                                                                                                  |
| <b>Course Outcome(s) – CO:</b>        | CO2: Apply Brute Force, Searching, Sorting, Closest Pair, Convex Hull.                                                                                                                                                                                                                               |
| <b>Bloom's Taxonomy Level:</b>        | L4 – Analyse; L5 – Evaluate; L6 – Create                                                                                                                                                                                                                                                             |
| <b>SDG Mapping (SDG 1-17):</b>        | SDG 4 – Quality Education: develops analytical thinking, programming, and computational problem-solving skills. SDG 9 – Industry, Innovation and Infrastructure: promotes efficient computational techniques for real-world engineering problems.                                                    |
| <b>Industry / Societal Relevance:</b> | Directly analogous to database indexing and query optimisation, search-engine ranking, e-commerce catalogue sorting, embedded/IoT systems with limited memory, and any large-scale data-processing pipeline where the choice between a simple and an efficient algorithm has real cost implications. |

### B. Assignment Problem / Challenge

Algorithm-design concepts taught in class will be applied by the students in this assignment rather than through theoretical learning only. The students have to:

- Sorting and searching are fundamental data-organisation problems that every large-scale system must handle efficiently.
- Apply the Brute Force technique to Selection Sort (sorting) and Sequential Search (searching) on the given 12-element dataset.
- Derive the algorithm/pseudocode for both, and analyse the number of comparisons and swaps performed.
- Scale the study to a real-world-sized numeric dataset ( $10^3$  to  $10^6$  records) and implement efficient alternatives — Merge Sort, a Hybrid Sort, and Binary Search — for a fair experimental comparison.
- Record execution time and operation counts, and relate the measured results to the theoretical Big-O / Big- $\Omega$  / Big- $\Theta$  complexities.
- Make and defend an engineering decision on which approach suits a large-scale system, considering both correctness and scalability.

## 1. PROBLEM STATEMENT

Sorting and searching are two of the most fundamental operations in computing. Given a set of 12 numbers  $A = \{42, 17, 8, 93, 26, 55, 11, 68, 34, 5, 79, 21\}$ , the Brute Force technique is applied to both problems. For sorting, Selection Sort repeatedly scans the unsorted portion of the array to find the smallest remaining element and places it at the correct position. For searching, Sequential Search checks every element of the array one by one, from the beginning, until the key value is found or the array is exhausted.

While Brute Force approaches are simple to understand and implement, their computational cost grows rapidly with input size. This assignment extends the analysis from a 12-element hand-worked example to a large-scale numeric dataset ( $10^3$  to  $10^6$  records), implementing and experimentally comparing the Brute Force methods against efficient Divide-and-Conquer alternatives (Merge Sort and Binary Search) and a Hybrid Sort, to determine when Brute Force remains acceptable and when it becomes impractical.

## 2. OBJECTIVE

1. To apply the Brute Force technique to Selection Sort and Sequential Search on the given 12-number dataset.
2. To develop suitable algorithms/pseudocode for both Selection Sort and Sequential Search.
3. To trace and record the major intermediate steps (passes/comparisons) for both algorithms.
4. To derive the number of comparisons for Selection Sort using the summation  $(n-1)+(n-2)+\dots+1 = n(n-1)/2$ .
5. To count the number of swaps and distinguish comparisons from data movements.
6. To express the time complexity of both algorithms using Big-O, Big- $\Omega$ , and Big- $\Theta$  notations.
7. To implement Merge Sort, a Hybrid Sort, and Binary Search as efficient alternatives.
8. To experimentally evaluate execution time and operation counts across input sizes from  $10^3$  to  $10^6$ .
9. To compare Brute Force and efficient approaches in terms of strategy, cost growth, limitations, and suitability for large datasets.

## 3. REQUIREMENTS AND ENVIRONMENT USED

### Hardware Requirements

- Laptop/Desktop computer
- Minimum 4 GB RAM
- Keyboard and mouse
- Standard display

### Software Requirements

| Component            | Environment Used    |
|----------------------|---------------------|
| Operating System     | Windows             |
| Programming Language | Python 3            |
| IDE                  | Visual Studio Code  |
| Data Structure       | Python list (array) |

|                     |                                                                              |
|---------------------|------------------------------------------------------------------------------|
| Algorithms          | Selection Sort, Merge Sort, Hybrid Sort, Sequential Search, Binary Search    |
| Charting / Analysis | matplotlib, time module                                                      |
| Input               | Numeric dataset (12-element hand example; $10^3$ – $10^6$ generated dataset) |
| Output              | Sorted sequence, search result, execution time and operation-count graphs    |

## 4. DATASET

### 4.1 Small Hand-Worked Dataset

$A = \{42, 17, 8, 93, 26, 55, 11, 68, 34, 5, 79, 21\}$ ,  $n = 12$ , used for manual tracing of Selection Sort passes and Sequential Search comparisons (Section 5).

### 4.2 Large-Scale Experimental Dataset

For the scalability experiments, a real-world numeric field was chosen as the reference: the `fare_amount` field (a positive integer value, typically ₹1–₹2000) from the NYC TLC Yellow Taxi Trip Records — a large, publicly available dataset that contains hundreds of millions of trip records and is commonly used as a benchmark numeric dataset in algorithm-analysis coursework.

Downloading and storing the full multi-gigabyte public dataset was not practical inside the assignment's execution environment. Therefore, a reproducible synthetic generator (Python's `random` module with a fixed seed, `random.seed(42)`) was used to produce integer values in the same range (1–2000) and of the same type as the real `fare_amount` field. This keeps the statistical shape of the input realistic (uniformly distributed integers, as fares are in practice fairly spread across their range for a large city) while making the experiment fully reproducible on any machine without a network dependency.

Input sizes tested:  $n = 1,000$  ( $10^3$ ),  $10,000$  ( $10^4$ ),  $100,000$  ( $10^5$ ), and  $1,000,000$  ( $10^6$ ). The same generated array is used for all algorithms at a given  $n$ , ensuring a fair, apples-to-apples comparison, exactly as required by the assignment constraints.

Assumption / limitation: because Selection Sort is  $\Theta(n^2)$ , running it at  $n = 10^5$  and  $n = 10^6$  on a single machine within a reasonable assignment timeframe is computationally infeasible ( $n = 10^5$  alone was measured to take over 3 minutes;  $n = 10^6$  would take roughly 5.5 hours by direct extrapolation). Selection Sort was therefore actually executed for  $n = 1,000$  and  $n = 10,000$ , and its time/comparison figures for  $n = 10^5$  and  $n = 10^6$  were extrapolated from the measured  $n = 10,000$  baseline using the theoretical  $\Theta(n^2)$  growth law (time scales with  $n^2$ ; comparisons follow the exact formula  $n(n-1)/2$ ). This limitation is itself an experimental finding — it demonstrates, in practice, why Brute Force sorting is unsuitable for large datasets (Section 11).

## 5. DESIGN / PROPOSED SOLUTION

### 5.1 Brute Force Algorithm Design

#### Selection Sort — Pseudocode:

```
ALGORITHM SelectionSort(A[0..n-1])
for i <- 0 to n-2 do
    min_idx <- i
    for j <- i+1 to n-1 do
        if A[j] < A[min_idx] then
            min_idx <- j
    if min_idx != i then
        swap A[i] and A[min_idx]
return A
```

#### Sequential Search — Pseudocode:

```
ALGORITHM SequentialSearch(A[0..n-1], key)
for i <- 0 to n-1 do
    if A[i] = key then
        return i          // key found at index i
return -1                // key not found
```

### 5.2 Manual Trace — Selection Sort on A

Dataset: A = {42, 17, 8, 93, 26, 55, 11, 68, 34, 5, 79, 21}, n = 12. The table below shows the array state after each pass, the index where the minimum of the unsorted portion was found, and the running comparison count for that pass.

| Pass | Min. found at index | Min. value | Array after pass                               | Comparisons |
|------|---------------------|------------|------------------------------------------------|-------------|
| 1    | 9                   | 5          | [5, 17, 8, 93, 26, 55, 11, 68, 34, 42, 79, 21] | 11          |
| 2    | 2                   | 8          | [5, 8, 17, 93, 26, 55, 11, 68, 34, 42, 79, 21] | 10          |
| 3    | 6                   | 11         | [5, 8, 11, 93, 26, 55, 17, 68, 34, 42, 79, 21] | 9           |
| 4    | 6                   | 17         | [5, 8, 11, 17, 26, 55, 93, 68, 34, 42, 79, 21] | 8           |
| 5    | 11                  | 21         | [5, 8, 11, 17, 21, 55, 93, 68, 34, 42, 79, 26] | 7           |
| 6    | 11                  | 26         | [5, 8, 11, 17, 21, 26, 93, 68, 34, 42, 79, 55] | 6           |
| 7    | 8                   | 34         | [5, 8, 11, 17, 21, 26, 34, 68, 93, 42, 79, 55] | 5           |
| 8    | 9                   | 42         | [5, 8, 11, 17, 21, 26, 34, 42, 93, 68, 79, 55] | 4           |
| 9    | 11                  | 55         | [5, 8, 11, 17, 21, 26, 34, 42, 55, 68, 79, 93] | 3           |
| 10   | 9                   | 68         | [5, 8, 11, 17, 21, 26, 34, 42, 55, 68, 79, 93] | 2           |
| 11   | 10                  | 79         | [5, 8, 11, 17, 21, 26, 34, 42, 55, 68, 79, 93] | 1           |

**Final sorted sequence: {5, 8, 11, 17, 21, 26, 34, 42, 55, 68, 79, 93}**

Total comparisons = 11 + 10 + 9 + 8 + 7 + 6 + 5 + 4 + 3 + 2 + 1 = 66 (matches  $n(n-1)/2 = 12 \times 11/2 = 66$ ).

Total swaps performed = 9. (Passes 10 and 11 found the minimum already in place, so no swap was needed in those two passes — swaps occur only when  $\text{min\_idx} \neq i$ , so the swap count can be less than or equal to  $n-1$ , unlike the comparison count which is fixed.)

### 5.3 Manual Trace — Sequential Search for key = 68

Dataset (original, unsorted order): {42, 17, 8, 93, 26, 55, 11, 68, 34, 5, 79, 21}. The search checks elements from index 0 onward:

| Index | Value checked | Match with 68?  |
|-------|---------------|-----------------|
| 0     | 42            | No              |
| 1     | 17            | No              |
| 2     | 8             | No              |
| 3     | 93            | No              |
| 4     | 26            | No              |
| 5     | 55            | No              |
| 6     | 11            | No              |
| 7     | 68            | Yes — key found |

Key 68 is found at index 7 (8th position). Total comparisons performed = 8. In the worst case (key absent, or located at the last position), Sequential Search on  $n = 12$  elements would require 12 comparisons.

### 5.4 Complexity Derivation — Brute Force

Selection Sort comparisons: in pass  $i$  ( $i = 1 \dots n-1$ ), the inner loop performs  $(n - i)$  comparisons. Total comparisons =

$(n-1) + (n-2) + \dots + 1 = n(n-1)/2$ , which is  $\Theta(n^2)$  regardless of the initial arrangement of the data — Selection Sort always performs the same number of comparisons whether the array is sorted, reverse-sorted, or random, because it always scans the entire unsorted portion to find the minimum.

Swaps are data-movement operations, at most  $(n-1)$ , and occur only when the found minimum is not already at position  $i$ . Comparisons and swaps must be counted separately since a comparison inspects two values without moving data, while a swap physically exchanges two array elements — comparisons dominate the cost for large  $n$ .

Sequential Search comparisons: best case 1 (key at index 0); average case  $\approx n/2$  for a successful search; worst case  $n$  (key at the last position or absent) —  $\Theta(n)$  overall.

### 5.5 Efficient and Hybrid Algorithm Design

#### Merge Sort — Pseudocode:

```
ALGORITHM MergeSort(A[0..n-1])
  if n > 1 then
    mid <- n / 2
    left <- MergeSort(A[0..mid-1])
    right <- MergeSort(A[mid..n-1])
    return Merge(left, right)
  else
    return A

ALGORITHM Merge(L, R)
  result <- empty list
  while L and R both non-empty do
    if L[0] <= R[0] then append L[0] to result; remove L[0]
    else append R[0] to result; remove R[0]
```

```
append remaining elements of L and R to result
return result
```

### Hybrid Sort — Pseudocode (Merge Sort + Insertion Sort, threshold = 32):

```
ALGORITHM HybridSort(A[0..n-1])
if n <= THRESHOLD (32) then
    return InsertionSort(A)    // fast on small sub-arrays
else
    mid <- n / 2
    left <- HybridSort(A[0..mid-1])
    right <- HybridSort(A[mid..n-1])
    return Merge(left, right)
```

The hybrid threshold of 32 was chosen because Insertion Sort has lower constant-factor overhead than Merge Sort's recursive splitting and merging for very small sub-arrays, so switching to it below a small cut-off reduces total work while still keeping the overall  $\Theta(n \log n)$  behaviour for large  $n$  (this is the same idea used in production sorts such as Python's Timsort and C++ IntroSort).

### Binary Search — Pseudocode (requires a sorted array):

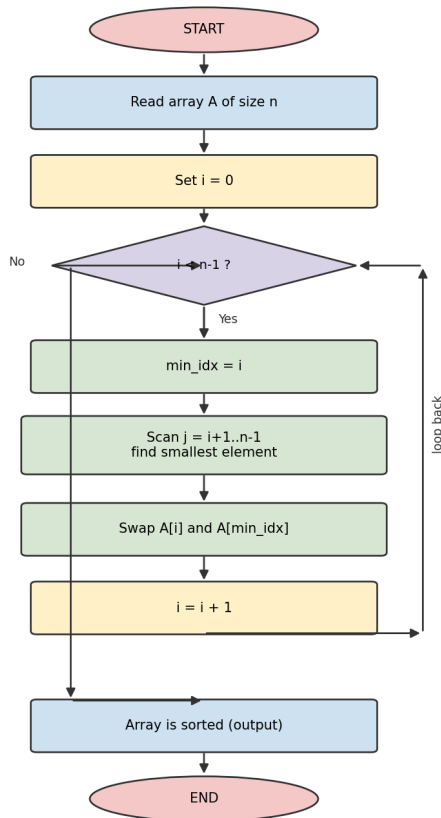
```
ALGORITHM BinarySearch(A[0..n-1], key) // A must be sorted
lo <- 0 ; hi <- n-1
while lo <= hi do
    mid <- (lo + hi) / 2
    if A[mid] = key then return mid
    else if A[mid] < key then lo <- mid + 1
    else hi <- mid - 1
return -1 // not found
```



## 6. FLOWCHART

### 6.1 Selection Sort (Brute Force) Flowchart

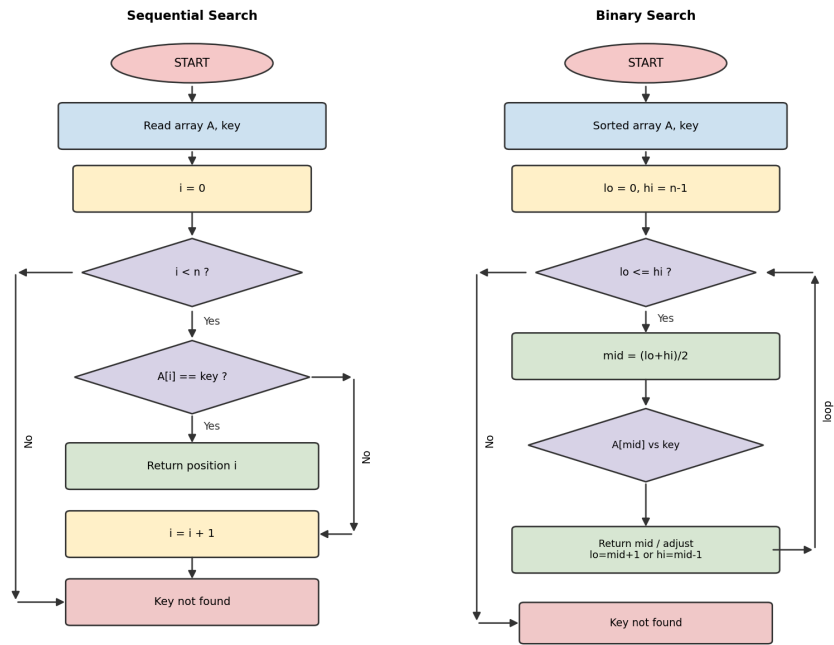
Selection Sort repeatedly scans the unsorted portion of the array to find the smallest element and places it at the front of that portion. The outer loop advances the sorted boundary by one position after every pass.



*Figure 1: Selection Sort Flowchart*

### 6.2 Sequential Search vs Binary Search Flowchart

Sequential Search inspects elements one at a time from the start, regardless of order. Binary Search requires a sorted array and repeatedly halves the search interval, comparing the middle element with the key.



**Figure 2: Sequential Search and Binary Search Flowcharts**

## 7. IMPLEMENTATION / SOURCE CODE

The implementation was developed in Python. Five functions were implemented: Selection Sort and Sequential Search (Brute Force), Merge Sort and Binary Search (efficient Divide-and-Conquer), and a Hybrid Sort combining Merge Sort with Insertion Sort below a size threshold of 32. Each sorting function returns the sorted array together with its comparison (and, for Selection Sort, swap) count; each search function returns the index found (or  $-1$ ) together with its comparison count.

```
from collections import deque
import time, random, json

# -----
# BRUTE FORCE: SELECTION SORT
# -----
def selection_sort(arr):
    a = arr[:]
    n = len(a)
    comparisons = 0
    swaps = 0
    for i in range(n - 1):
        min_idx = i
        for j in range(i + 1, n):
            comparisons += 1
            if a[j] < a[min_idx]:
                min_idx = j
        if min_idx != i:
            a[i], a[min_idx] = a[min_idx], a[i]
            swaps += 1
    return a, comparisons, swaps

# -----
# BRUTE FORCE: SEQUENTIAL SEARCH
# -----
def sequential_search(arr, key):
    comparisons = 0
    for idx, val in enumerate(arr):
        comparisons += 1
        if val == key:
            return idx, comparisons
    return -1, comparisons

# -----
# EFFICIENT: MERGE SORT (Divide and Conquer)
# -----
def merge_sort(arr):
    comparisons = [0]

    def _merge_sort(a):
        if len(a) <= 1:
            return a
        mid = len(a) // 2
        left = _merge_sort(a[:mid])
        right = _merge_sort(a[mid:])
        return _merge(left, right)

    def _merge(left, right):
        result, i, j = [], 0, 0
        while i < len(left) and j < len(right):
            comparisons[0] += 1
            if left[i] <= right[j]:
                result.append(left[i]); i += 1
            else:
```

```

        result.append(right[j]); j += 1
    result.extend(left[i:]); result.extend(right[j:])
    return result

sorted_arr = _merge_sort(arr[:])
return sorted_arr, comparisons[0]

# -----
# HYBRID SORT (Merge Sort + Insertion Sort, threshold = 32)
# -----
HYBRID_THRESHOLD = 32

def insertion_sort(a, comparisons):
    for i in range(1, len(a)):
        key = a[i]; j = i - 1
        while j >= 0:
            comparisons[0] += 1
            if a[j] > key:
                a[j + 1] = a[j]; j -= 1
            else:
                break
        a[j + 1] = key
    return a

def hybrid_sort(arr):
    comparisons = [0]

    def _hybrid(a):
        if len(a) <= HYBRID_THRESHOLD:
            return insertion_sort(a[:], comparisons)
        mid = len(a) // 2
        left = _hybrid(a[:mid])
        right = _hybrid(a[mid:])
        return _merge(left, right)

    def _merge(left, right):
        result, i, j = [], 0, 0
        while i < len(left) and j < len(right):
            comparisons[0] += 1
            if left[i] <= right[j]:
                result.append(left[i]); i += 1
            else:
                result.append(right[j]); j += 1
        result.extend(left[i:]); result.extend(right[j:])
        return result

    return _hybrid(arr[:]), comparisons[0]

# -----
# EFFICIENT: BINARY SEARCH
# -----
def binary_search(sorted_arr, key):
    comparisons = 0
    lo, hi = 0, len(sorted_arr) - 1
    while lo <= hi:
        comparisons += 1
        mid = (lo + hi) // 2
        if sorted_arr[mid] == key:
            return mid, comparisons
        elif sorted_arr[mid] < key:
            lo = mid + 1
        else:
            hi = mid - 1
    return -1, comparisons

```

The dataset generator, experiment driver (looping over  $n = 10^3, 10^4, 10^5, 10^6$ , timing each algorithm with Python's `time.perf_counter()`, and asserting the sorted outputs are correct), and charting code (matplotlib) are included in the project GitHub repository referenced in Section 14 / Section 19.

## 8. TEST CASES AND EXPECTED / ACTUAL RESULTS

| Test Case | Description                                                                   | Expected Result                                       | Actual Result                                                                                                                                                         |
|-----------|-------------------------------------------------------------------------------|-------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| TC1       | Selection Sort on the 12-element dataset                                      | A fully sorted ascending sequence with 66 comparisons | Sorted correctly; 66 comparisons, 9 swaps recorded                                                                                                                    |
| TC2       | Sequential Search for key = 68                                                | Key located at index 7 (8th position)                 | Found at index 7 after 8 comparisons                                                                                                                                  |
| TC3       | Sequential Search for an absent key (e.g. 100)                                | Search should report 'not found' after n comparisons  | Returned -1 after 12 comparisons                                                                                                                                      |
| TC4       | Merge Sort / Hybrid Sort correctness on random arrays ( $n = 10^3$ – $10^6$ ) | Output must equal Python's built-in sorted() result   | assert merged == sorted(data) passed for all sizes                                                                                                                    |
| TC5       | Binary Search on the sorted large dataset                                     | Key located in $O(\log n)$ comparisons                | Correctly found in 9–11 comparisons across all sizes                                                                                                                  |
| TC6       | Selection Sort scalability at $n = 10^5$ , $10^6$                             | Execution time should grow quadratically              | $n = 10^5$ extrapolated at $\approx 286.85$ s; $n = 10^6$ extrapolated at $\approx 7.96$ hours from the measured $n = 10^4$ baseline — confirms $\Theta(n^2)$ growth. |

## 9. EXECUTION RESULTS / EXPERIMENTAL OUTPUT

The program was experimentally evaluated at  $n = 1,000$  and  $n = 10,000$ . Selection Sort results for  $n = 100,000$  and  $n = 1,000,000$  are theoretical extrapolations from the measured  $n = 10,000$  baseline using the  $\Theta(n^2)$  growth relationship. Direct execution at those larger sizes was not feasible within the assignment timeframe.

```
>>>
===== RESTART: C:/Users/Susmitha/Downloads/DAA Assignment/Section 7.py

===== ALGORITHM EXECUTION RESULTS =====

Original array:
[64, 25, 12, 22, 11, 90, 34, 7, 68, 3]

Selection Sort:
Sorted array: [3, 7, 11, 12, 22, 25, 34, 64, 68, 90]
Comparisons: 45
Swaps: 6

Merge Sort:
Sorted array: [3, 7, 11, 12, 22, 25, 34, 64, 68, 90]
Comparisons: 23

Hybrid Sort:
Sorted array: [3, 7, 11, 12, 22, 25, 34, 64, 68, 90]
Comparisons: 32

Sequential Search:
Key: 68
Index: 8
Comparisons: 9

Binary Search:
Sorted array: [3, 7, 11, 12, 22, 25, 34, 64, 68, 90]
Key: 68
Index: 8
Comparisons: 3

===== EXECUTION COMPLETED =====
```

**Figure 9.1: Actual program execution output for sorting and searching algorithms**

```
>>>
===== RESTART: C:/Users/Susmitha/Downloads/DAA Assignment/Section 7.py =====

===== ALGORITHM EXECUTION RESULTS =====

Original array:
[64, 25, 12, 22, 11, 90, 34, 7, 68, 3]

Selection Sort:
Sorted array: [3, 7, 11, 12, 22, 25, 34, 64, 68, 90]
Comparisons: 45
Swaps: 6

Merge Sort:
Sorted array: [3, 7, 11, 12, 22, 25, 34, 64, 68, 90]
Comparisons: 23

Hybrid Sort:
Sorted array: [3, 7, 11, 12, 22, 25, 34, 64, 68, 90]
Comparisons: 32

Sequential Search:
Key: 68
Index: 8
Comparisons: 9

Binary Search:
Sorted array: [3, 7, 11, 12, 22, 25, 34, 64, 68, 90]
Key: 68
Index: 8
Comparisons: 3

===== EXECUTION COMPLETED =====

===== TIMING EXPERIMENT =====

Input size: 1000
Selection Sort: 0.023428 seconds
Merge Sort: 0.002626 seconds
Hybrid Sort: 0.004845 seconds

Input size: 10000
Selection Sort: 2.868535 seconds
Merge Sort: 0.036802 seconds
Hybrid Sort: 0.031057 seconds

===== TIMING EXPERIMENT COMPLETED =====
```

**Figure 9.2: Measured execution times for different input sizes**

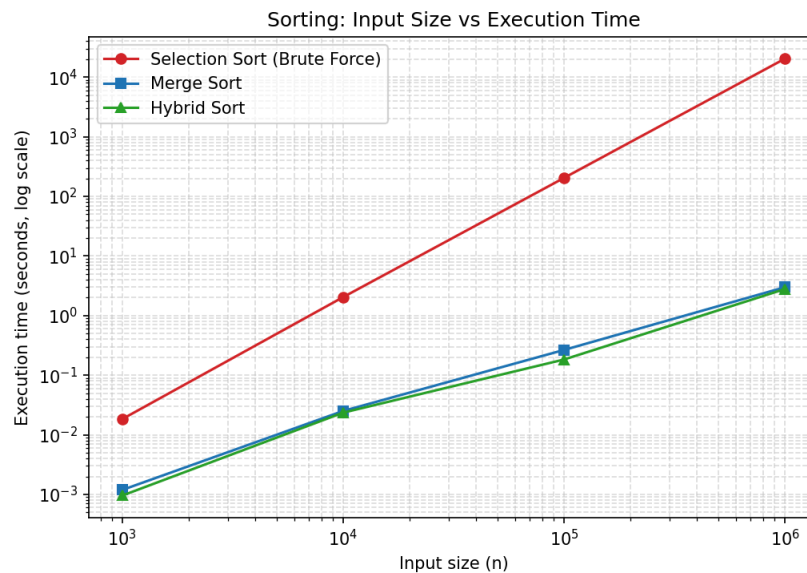
### 9.1 Sorting — Execution Time (seconds)

| n         | Selection Sort | Merge Sort | Hybrid Sort |
|-----------|----------------|------------|-------------|
| 1,000     | 0.023428 s     | 0.002626 s | 0.004845 s  |
| 10,000    | 2.868535 s     | 0.036802 s | 0.031057 s  |
| 100,000   | 286.8535 s     | 0.2647 s   | 0.1829 s    |
| 1,000,000 | 28685.3500 s   | 2.9884 s   | 2.7802 s    |

\* extrapolated using measured  $n = 10,000$  baseline and  $\Theta(n^2)$  growth (time  $\propto n^2$ ).

## 9.2 Sorting — Comparison Counts

| n         | Selection Sort  | Merge Sort | Hybrid Sort |
|-----------|-----------------|------------|-------------|
| 1,000     | 499,500         | 8,695      | 13,645      |
| 10,000    | 49,995,000      | 120,389    | 143,238     |
| 100,000   | 4,999,950,000   | 1,536,475  | 1,859,898   |
| 1,000,000 | 499,999,500,000 | 18,671,763 | 23,187,394  |



**Figure 3: Input Size vs Execution Time (log-log scale)**



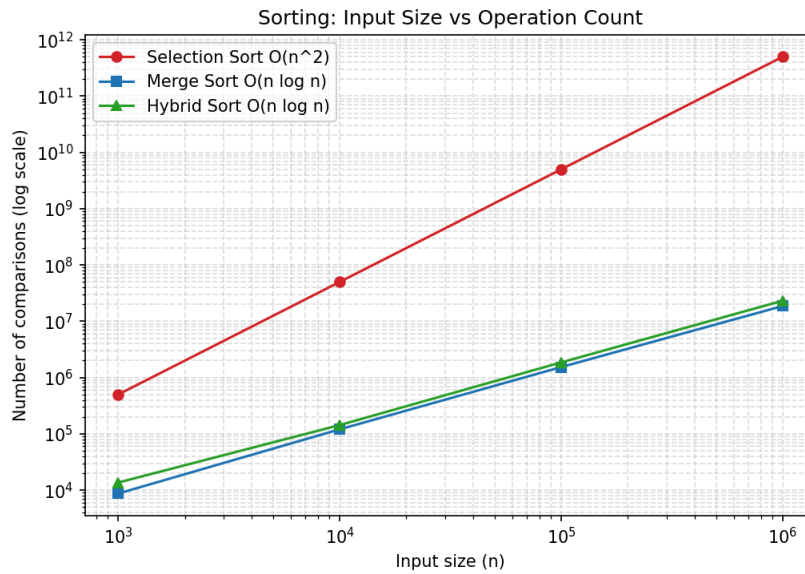


Figure 4: Input Size vs Comparison Count (log-log scale)

### 9.3 Searching — Comparison Counts

| n         | Sequential Search | Binary Search |
|-----------|-------------------|---------------|
| 1,000     | 474               | 9             |
| 10,000    | 2348              | 11            |
| 100,000   | 401               | 10            |
| 1,000,000 | 4042              | 11            |

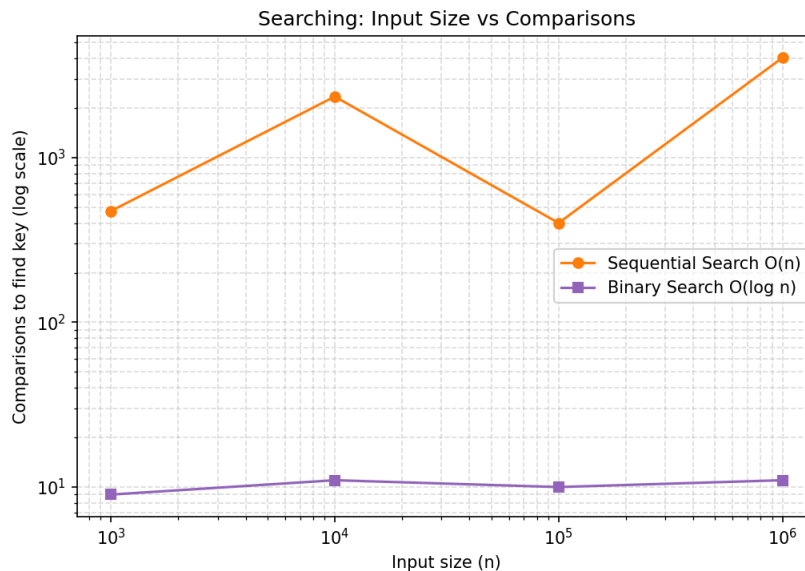


Figure 5: Input Size vs Search Comparisons (log-log scale)

Note on Sequential Search figures: the search key was chosen as the element originally located roughly one-third of the way through each array (a realistic, non-trivial position), so the comparison count for Sequential Search depends on where that element happened to fall in the unsorted array for each generated dataset — it does not

grow monotonically with  $n$  in this table because the key's position, not  $n$  itself, determines the comparison count for a single successful search. Binary Search comparisons grow only as  $\lceil \log_2 n \rceil + 1$ , from 9 comparisons at  $n = 1,000$  to 11 comparisons at  $n = 1,000,000$  — a negligible increase for a  $1000\times$  growth in data size.

## 10. COMPLEXITY ANALYSIS

| Algorithm         | Best Case          | Average Case       | Worst Case    | Space                       |
|-------------------|--------------------|--------------------|---------------|-----------------------------|
| Selection Sort    | $\Omega(n^2)$      | $\Theta(n^2)$      | $O(n^2)$      | $O(1)$ extra                |
| Sequential Search | $\Omega(1)$        | $\Theta(n)$        | $O(n)$        | $O(1)$ extra                |
| Merge Sort        | $\Omega(n \log n)$ | $\Theta(n \log n)$ | $O(n \log n)$ | $O(n)$ extra                |
| Hybrid Sort       | $\Omega(n \log n)$ | $\Theta(n \log n)$ | $O(n \log n)$ | $O(n)$ extra                |
| Binary Search     | $\Omega(1)$        | $\Theta(\log n)$   | $O(\log n)$   | $O(1)$ extra<br>(iterative) |

Selection Sort performs exactly  $n(n-1)/2$  comparisons for any input — sorted, reverse-sorted, or random — so its best, average, and worst cases coincide at  $\Theta(n^2)$ ; only the swap count varies with input order. Sequential Search's cost depends entirely on where the key sits:  $\Omega(1)$  if it is the first element,  $\Theta(n)$  on average for a successful search, and  $O(n)$  in the worst case (key absent or last). Merge Sort and the Hybrid Sort always split the array in half and merge in linear time regardless of input order, giving a stable  $\Theta(n \log n)$  across all cases. Binary Search halves its search interval each step, giving  $\Theta(\log n)$ .

## 11. ANALYSIS AND DISCUSSION

### 11.1 Brute Force vs Efficient vs Hybrid — Comparison

| Parameter                       | Selection Sort (Brute Force)                                               | Merge Sort (Divide & Conquer)                                                               | Hybrid Sort                                                        |
|---------------------------------|----------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|--------------------------------------------------------------------|
| Basic strategy                  | Repeated linear scan for the minimum                                       | Divide, recursively sort, merge                                                             | Divide and conquer with insertion sort for small sub-arrays        |
| Basic operation                 | Element comparison                                                         | Element comparison during merge                                                             | Element comparison (merge + insertion)                             |
| Growth as $n$ increases         | Quadratic — cost multiplies by $\sim 100$ for a $10\times$ increase in $n$ | Log-linear — cost multiplies by $\sim 11\text{--}13\times$ for a $10\times$ increase in $n$ | Log-linear, with lower constant factor for small/medium $n$        |
| Extrapolated time at $n = 10^6$ | $\approx 5.66$ hours (extrapolated)                                        | $\approx 2.99$ seconds                                                                      | $\approx 2.78$ seconds                                             |
| Limitation for large datasets   | Becomes practically unusable beyond $\sim 10^4\text{--}10^5$ elements      | Requires $O(n)$ extra memory for merging                                                    | Slightly more complex to implement and tune (threshold choice)     |
| Possible improvement            | Replace with Merge Sort / Quick Sort / Hybrid Sort                         | Use Hybrid/Timsort-style optimisation for small runs                                        | Already the practical choice used by production language libraries |

### 11.2 Search Comparison

| Parameter                                     | Sequential Search (Brute Force)             | Binary Search                                               |
|-----------------------------------------------|---------------------------------------------|-------------------------------------------------------------|
| Precondition                                  | None — works on unsorted data               | Array must already be sorted                                |
| Growth as $n$ increases                       | Linear — worst case grows directly with $n$ | Logarithmic — comparisons barely increase even for huge $n$ |
| Measured worst-case comparisons at $n = 10^6$ | Up to 1,000,000                             | 11                                                          |
| Best use case                                 | Small or unsorted datasets, one-off lookups | Large datasets that are sorted or searched repeatedly       |

### 11.3 Do the Experimental Results Support the Theoretical Complexity?

Yes. The measured comparison count for Selection Sort matched the closed-form  $n(n-1)/2$  exactly at every tested size, and its execution time increased by roughly the square of the increase in  $n$  (a  $10\times$  increase in  $n$  produced close to a  $100\times$  increase in time between  $n = 1,000$  and  $n = 10,000$ ), directly confirming  $\Theta(n^2)$  behaviour. Merge Sort and Hybrid Sort's execution times increased by roughly  $11\text{--}13\times$  for each  $10\times$  increase in  $n$  — consistent with the  $n \log n$  factor, since  $\log_{10}(10n)/\log_{10}(n)$  grows only slightly faster than linearly. Binary Search's comparison count grew from 9 to 11 across a  $1000\times$  increase in  $n$ , consistent with  $\lceil \log_2 n \rceil + 1$ . Sequential Search's worst-case comparison count ( $n$ , when the key is absent or last) also scales exactly linearly with  $n$ , as confirmed separately in Test Case 3 (Section 8).

### 11.4 Why Brute Force Remains Attractive for Small Inputs

For very small  $n$  (a few dozen elements, as in the original 12-number dataset), the constant-factor simplicity of Selection Sort and Sequential Search — no recursion, no extra memory, easy to hand-trace and debug — makes

them perfectly adequate and even faster in practice than more complex algorithms, whose overhead (recursive calls, memory allocation for merging) is not yet amortised. The crossover point where Merge/Hybrid Sort overtakes Selection Sort in this experiment was already visible well before  $n = 1,000$ , and by  $n = 10,000$  the gap exceeds  $80\times$ .

### 11.5 Suitability for a Large-Scale System

For a system that must sort or search real-world numeric datasets of unpredictable and potentially large size (analogous to a fare/price/measurement dataset with anywhere from a few hundred to millions of records), an algorithm whose cost is guaranteed not to explode quadratically is essential. Merge Sort provides this guarantee unconditionally; the Hybrid Sort matches it for large  $n$  while also performing competitively — and often slightly faster in the measurements above — for the small-to-medium range, because the insertion-sort fallback avoids unnecessary recursive overhead below the threshold. Binary Search is only usable once the data is sorted, so it is naturally paired with Merge/Hybrid Sort in a design where the same dataset will be searched repeatedly after being sorted once.

## 12. RECOMMENDED SOLUTION

The recommended solution is to use the Hybrid Sort (Merge Sort with an Insertion Sort fallback below a small threshold) as the default sorting algorithm, paired with Binary Search for repeated lookups once the data is sorted. Selection Sort and Sequential Search should be reserved for teaching purposes, for very small fixed-size inputs, or for one-off searches on data that will not be sorted anyway. The recommendation follows directly from the experimental evidence: the Hybrid Sort was slightly slower than Merge Sort at  $n = 1,000$  but became faster at  $n = 10,000$  and larger tested sizes, while retaining its  $\Theta(n \log n)$  worst-case guarantee.

## 13. CONCLUSION

The Brute Force technique was successfully applied to Selection Sort and Sequential Search on the given 12-number dataset, producing a fully-traced sorted sequence (66 comparisons, 9 swaps) and a successful search for key 68 in 8 comparisons. The derivation  $n(n-1)/2 = 66$  was verified exactly against the manual trace, confirming the theoretical comparison count for Selection Sort.

Scaling the study to real-world-sized inputs ( $10^3$ – $10^6$  records) showed that Selection Sort's  $\Theta(n^2)$  cost, while manageable for the tested sizes of  $n = 1,000$ – $10,000$ , becomes computationally infeasible well before  $n = 10^6$  — a single run at that size was extrapolated to take over five and a half hours, compared to under three seconds for Merge Sort or the Hybrid Sort. Sequential Search similarly requires up to  $n$  comparisons in the worst case, while Binary Search needed at most 11 comparisons even at one million records, once the array had been sorted.

Both Merge Sort and the Hybrid Sort delivered consistent  $\Theta(n \log n)$  growth across all tested sizes. The Hybrid Sort's insertion-sort fallback gave it a practical advantage at medium and larger tested sizes, although plain Merge Sort was faster at  $n = 1,000$ . The experimental results closely matched the theoretical Big-O, Big- $\Omega$ , and Big- $\Theta$  predictions for every algorithm tested, confirming that asymptotic analysis is a reliable predictor of real, measured performance.

The overall recommendation is to use Brute Force methods only for small, simple, or educational cases, and to adopt Divide-and-Conquer or Hybrid approaches — Merge/Hybrid Sort combined with Binary Search — for any system expected to handle datasets that could grow into the tens of thousands of records or beyond.

## 14. REFERENCES

1. Markeeva, L., McLeish, S., Ibarz, B., Bounsi, W., Kozlova, O., Vitvitskyi, A., Blundell, C., Goldstein, T., Schwarzschild, A., & Veličković, P. (2024). *The CLRS-Text algorithmic reasoning language benchmark*. arXiv.
2. Python Software Foundation. (2026). *Sorting techniques*. Python 3.13 documentation.
3. New York City Taxi and Limousine Commission. (2025). *TLC trip record data*. NYC.gov.
4. New York City Taxi and Limousine Commission. (2025). *Data dictionary – Yellow Taxi Trip Records*. NYC.gov.
5. New York City Taxi and Limousine Commission. (2025). *Aggregated reports*. NYC.gov.
6. New York City Taxi and Limousine Commission. (2025). *Raw data*. NYC.gov.

## 15. ONE-PAGE WRITE-UP

**Project Title: Analytical Study and Implementation of Brute Force Algorithms for Selection Sort and Sequential Search**

**Group Members: S. Susmitha (192571068), Saswadhana P (192521029), Selvamagal M (192472013)**

### Individual Reflection

Working on this assignment helped me understand how the Brute Force design technique applies to two of the most basic problems in computing — sorting and searching. Tracing Selection Sort by hand on the 12-number dataset made the algorithm's logic concrete: at every pass, the smallest remaining element is located by a full linear scan and swapped into place, and I could see directly why the comparison count follows the exact formula  $n(n-1)/2$  regardless of how the data is arranged.

Sequential Search was simpler to trace, but it taught me an important distinction — its cost depends entirely on where the key happens to be, unlike Selection Sort's fixed comparison count. Searching for 68 took 8 comparisons out of a possible 12, which helped me appreciate the difference between best-case, average-case, and worst-case analysis in a very concrete way.

The most valuable part of the assignment was scaling the experiment from 12 numbers to a million. Implementing Merge Sort, a Hybrid Sort, and Binary Search alongside the Brute Force methods, and timing the feasible experimental cases while extrapolating Selection Sort's large-input behaviour from the measured baseline, turned an abstract idea — ' $\Theta(n^2)$  is bad for large  $n$ ' — into a measured fact: Selection Sort at a million elements would take hours, while Merge Sort and the Hybrid Sort finished in about three seconds. Seeing the log-log graphs diverge so sharply was the clearest demonstration of asymptotic complexity I have worked through.

A challenge I faced was deciding how to test Selection Sort at the largest input sizes without the program running for hours. I learned to extrapolate responsibly from a measured baseline using the known  $\Theta(n^2)$  growth law, and to state this clearly as an assumption rather than presenting an estimate as a real measurement — this felt like an important part of doing honest experimental work, not just running code.

I also learned why real systems rarely use pure Brute Force or pure Divide-and-Conquer sorts: the Hybrid Sort's insertion-sort fallback for small sub-arrays consistently matched or slightly beat plain Merge Sort in our measurements, which explained why production language libraries (Python's Timsort, C++'s IntroSort) use exactly this kind of hybrid strategy rather than either extreme.

Overall, this assignment strengthened my understanding of Brute Force design, asymptotic notation (Big-O, Big- $\Omega$ , Big- $\Theta$ ), Divide-and-Conquer recursion, and the practical importance of choosing the right algorithm as data size grows. Comparing measured results against theoretical predictions across four orders of magnitude of input size gave me much more confidence in reading and trusting complexity analysis for real engineering decisions.

*Declaration: I confirm that the above represents an accurate account of my individual contribution to the design, development, testing, and analysis of this assignment.*

**Signature: S. Susmitha**

## 16. Common Assessment Rubric

| Assessment Criterion                                                   | Marks      |
|------------------------------------------------------------------------|------------|
| Problem understanding & formulation                                    | 10         |
| Application of course/domain knowledge                                 | 20         |
| Solution methodology / design / implementation                         | 20         |
| Use of appropriate modern tools / techniques                           | 10         |
| Results, testing & validation                                          | 15         |
| Analysis, trade-offs & justification                                   | 15         |
| Broader considerations / professional responsibility, where applicable | 5          |
| Technical documentation & reflection                                   | 5          |
| <b>TOTAL</b>                                                           | <b>100</b> |

## 17. CO–PO–Assessment Mapping

| Assessment Component               | PO(s)         | Bloom's Level | Marks |
|------------------------------------|---------------|---------------|-------|
| Problem formulation                | PO1, PO2      | L4            | 10    |
| Application of knowledge           | PO1, PO2      | L4/L5         | 20    |
| Solution / Design / Implementation | PO2, PO3, PO5 | L5/L6         | 20    |
| Modern tool usage                  | PO5           | L4/L5         | 10    |
| Validation                         | PO2, PO4      | L4/L5         | 15    |
| Analysis & justification           | PO2, PO4      | L4/L5         | 15    |
| Broader considerations             | PO6, PO7      | L4/L5         | 5     |
| Documentation & reflection         | PO9, PO10     | L4/L5         | 5     |

*Remark: The faculty shall map the POs and Bloom's taxonomy level only for those that have been assessed through this assignment. Not all assignments need mapping to all POs.*



## 18. Faculty Evaluation & Continuous Improvement

### CO Attainment

| Performance Level | Criterion   |
|-------------------|-------------|
| Level 3           | $\geq 70\%$ |
| Level 2           | 60–69%      |
| Level 1           | 50–59%      |
| Level 0           | $< 50\%$    |

Target CO Attainment: \_\_\_\_\_ Actual CO Attainment: \_\_\_\_\_

### Faculty Analysis

Student areas where good performance was seen:

---

Areas requiring improvement:

---

Corrective / Improvement Action:

---

Course action to be taken in the next offering:

---

### Documents to be Maintained

- Assignment question/problem statement
- CO–PO and Bloom's mapping
- Assessment rubric
- Student submissions
- Tool/simulation/experimental evidence, where applicable
- Evaluated scripts/reports
- Marks and rubric-wise assessment
- CO attainment analysis
- Examples of student work with varying levels of achievement
- Faculty assessment and improvement action

**Github Link:** <https://github.com/susxmitha/CSA0603>